

Overview

Before modernizing a .NET Framework application, it's important to understand the codebase structure, dependencies, and potential technical debt. In this section, you'll use the **AWS/comprehensive-codebase-analysis** AWS-managed transformation to generate a detailed analysis of the **Wild Rydes** sample application.

About the Wild Rydes Application

Wild Rydes is a ride-sharing web application built on a legacy .NET Framework stack. It allows users to request unicorn rides, manage user profiles, and browse available unicorns with location services.

Layer	Technology
Language	C#
Framework	.NET Framework 4.8.1
Web Framework	ASP.NET MVC 5.3 (Razor Views)
ORM	Entity Framework 6.5.1
Database	SQL Server LocalDB (MDF file)
AWS SDKs	AWSSDK.CognitoIdentity, AWSSDK.LocationService, AWSSDK.SecurityToken
Frontend	Bootstrap 5.3, jQuery 3.7, Modernizr
Serialization	Newtonsoft.Json 13.0, System.Text.Json 8.0
Package Management	NuGet (packages.config)

Project structure:

```
wildrydes/
├── sourceCode/
│   ├── wildrydes.net.sln          # Solution file
│   └── wildrydes.net/
│       ├── Controllers/          # HomeController, RideController, UnicornController, UserController
│       ├── Models/               # RideModel, UnicornModel, UserModel
│       ├── Views/                # Razor views (Home, Ride, Unicorn, User, Shared)
│       ├── Context/              # EF DbContext
│       ├── Migrations/           # EF database migrations
│       ├── App_Start/            # MVC route/bundle configuration
│       ├── Content/              # CSS and static assets
│       ├── Scripts/              # JavaScript files
│       ├── Web.config             # App configuration (SQL Server, AWS settings)
│       ├── packages.config        # NuGet dependencies (net481)
│       ├── wildrydes.net.csproj   # Project file
│       └── references/            # Reference assemblies and packages
└── references/
```

Modernization goal: Transform this .NET Framework 4.8.1 application to cross-platform .NET 8, enabling it to run on Linux, containers, and serverless platforms — reducing Windows licensing costs and improving performance.

The `AWS/comprehensive-codebase-analysis` transformation performs deep static analysis to generate hierarchical, cross-referenced documentation covering architecture, business logic, and technical debt — giving you a clear picture before starting the modernization.

Reference

Official documentation: [AWS-Managed Transformations](#)

EXERCISE 1: PREPARE THE SAMPLE APPLICATION

1. Verify you are in the workshop directory

```
pwd
```

Expected result

```
/Workshop
```

2. Download the sample .NET Framework application

```
curl -fsSL -o wildrydes.zip https://leelapornudom.com/public/wildrydes.zip
```

3. Unzip the application

```
unzip wildrydes.zip
```

4. Navigate into the project directory

```
cd wildrydes
```

5. Initialize a git repository (required by ATX)

```
git init && git add . && git commit -m "Initial commit"
```

6. Verify the project structure

```
ls -la
```

EXERCISE 2: RUN CODEBASE ANALYSIS

1. Run the `AWS/comprehensive-codebase-analysis` transformation interactively

```
atx custom def exec -n AWS/comprehensive-codebase-analysis -p . -t
```

Flags

- n — Name of the AWS-managed transformation
- p . — Path to the code repository (current directory)
- t — Trust all tools (no tool prompts)

2. Wait for the analysis to complete

The agent will analyze the codebase and generate documentation. This typically takes **30 to 60 minutes** depending on the size of the application.

You should see output similar to:

```
Region: us-east-1

All tools are now trusted. ATX will execute tools without asking for confirmation.

You are using an AWS Managed transformation.

Conversation log: /home/participant/.aws/atx/custom/<session_id>/logs/<timestamp>-conversation.log
Monitor progress: tail -f /home/participant/.aws/atx/custom/<session_id>/logs/<timestamp>-conversation.log

If interrupted, you can resume this conversation by running: atx --conversation-id <session_id>

> I'll begin orchestrating the transformation. Let me start with Phase 1: Planning.

I'll start the planning process for this Comprehensive Codebase Analysis transformation.
Let me begin by asking for your input.

**Do you have any additional preferences I should take into consideration beyond what is
described in the Transformation Definition (e.g., specific areas of focus, particular
concerns about the codebase, etc.)?**

Please let me know, or if you'd like me to proceed with the standard analysis approach,
just say so.
```

3. The ATX agent may ask you several questions during the analysis (e.g., preferences, areas of focus, confirmation to proceed). Simply respond with `go` each time to let it continue with the default approach.

```
go
```

Tip

The agent may prompt you multiple times. Just keep responding with `go` until the analysis completes.

4. Once the analysis is done, you should see the validation result

```
> Validation successful - All exit criteria passed. Complete validation summary has been
written to ~/.aws/atx/custom/<session_id>/artifacts/validation_summary.md.

Do you have any feedback or questions about the transformation execution?
```

5. Type `/quit` to exit the ATX session

```
/quit
```

6. Review the generated documentation

```
ls -la ATXDocumentation/
```

The transformation generates a comprehensive documentation suite from static analysis of the codebase (~1,645 lines of code across 27 files):

```
ATXDocumentation/
├── README.md          # Master table of contents and cross-reference index
├── project-overview.md # Tech stack, metadata, deployment model, source tree
├── technical-debt-report.md # Executive summary with AWS transformation recommendations
├── architecture/      # System-level design (layers, components, dependencies, patterns)
├── behavior/          # Runtime behavior (business logic, workflows, error handling)
├── reference/         # Developer reference (program structure, APIs, data models)
├── analysis/          # Code quality and security assessments
├── technical-debt/     # Actionable debt tracking with remediation plan
├── diagrams/          # Visual representations (Mermaid/text-based)
├── specialized/       # Domain-specific deep dives (DB schema, UI components)
└── migration/         # Modernization planning (7-phase plan, 40+ test cases)
```

Recommended reading order

- For modernization planning:** `technical-debt-report.md` → `architecture/system-overview.md` → `migration/component-order.md`
- For codebase onboarding:** `project-overview.md` → `architecture/` → `behavior/`

EXERCISE 3: REVIEW THE ANALYSIS RESULTS

1. Review the project overview

```
cat ATXDocumentation/project-overview.md
```

2. Review the technical debt report

```
cat ATXDocumentation/technical-debt-report.md
```

3. Review the architecture overview

```
cat ATXDocumentation/architecture/system-overview.md
```

4. Review the modernization plan

```
cat ATXDocumentation/migration/component-order.md
```

This provides a 7-phase migration plan based on the dependency hierarchy of the application.

5. Review the changes made by the transformation

```
git status
```

You should see the `ATXDocumentation/` folder listed as new untracked files added by the transformation.

What You've Learned

In this section, you:

- Understood the Wild Rydes application structure and its .NET Framework 4.8.1 tech stack
- Downloaded and prepared the sample application for ATX analysis
- Ran the `AWS/comprehensive-codebase-analysis` transformation using the ATX CLI
- Reviewed the generated documentation including project overview, technical debt report, architecture, and a 7-phase migration plan

This analysis gives you a solid understanding of the codebase before proceeding with the actual .NET modernization in the next section.